Architecture & Design

September 01, 1996

Email Print
Discuss Reprint

add to:

Del.icio.us Slashdot
Digg Y!
Google MyWeb
Spurl Blink
Furl

Algorithm Alley

(Page 5 of 5)

HATs: Hashed Array Trees

Fast variable-length arrays

Edward Sitarski

Edward is vice president of R&D at Numetrix Ltd. He can be reached at ed@ tor.numetrix.com.

Arrays are the most natural and convenient data structure for a great many applications. They provide constant access time to their elements and are the structure of choice for many practical algorithms. Most commercial C++ toolkits offer a variable-length array container class, and probably millions of C++ programs use variable-length arrays as their core data structures. Increasing the size of a variable-length array by appending to the end is a common operation; if we knew the size of the array at the start then there would be no reason to use a variable-length array.

I became painfully aware of the limitations of most implementations of variable-length arrays when investigating some code that read the results of a database query. There was no way to determine the number of rows returned from the query beforehand, and the code used a commercial variable-length array class to store the data. The profiler told me that the array class was doing an incredible amount of element copying when resizing. This copying was taking much more time than the slow data transfer from the database!

You might argue that another data structure, such as a linked list, would have been more appropriate for this application, but it was too difficult to fix all the old code. I decided to try to implement a better performing array class that had the same interface as the old one.

Many implementations of variable-length arrays leave much to be desired. Specifically, the function that appends to the end of the array can cause a large performance problem. Example 1

DEPARTMENTS
Home
Architecture & Design
C/C++
Database
Development Tools
Embedded Systems
High Performance Computing
Java
Mobility
Open Source
Security
Web Development
Windows/.NET
Play Sparkleball
Flipbook

shows just about the worst way to do it. There are two main reasons why this implementation is so bad.

- Excessive element copying. If you add 1000 elements to the array, the internal loop will copy the old elements into newTArray each time the array expands, a total of over 500,000 copy operations. This seemingly innocent code is doing about 500 times more assignments than necessary for the 1000 elements, and it only gets worse as the array grows. This implementation is O(N2) (the same time complexity as Bubblesort) and highly inefficient even for small arrays.
- Excessive memory usage. During the copy operation, both the existing array and the new array must be present in memory at the same time, doubling the memory requirement of the array. Worse, each time the array grows, it creates a memory fragment: the old array. This fragment cannot be reused the next time the array grows because the new array is larger and will not fit, as shown in Figure 1. This can increase the memory requirements by almost another factor of two.

It is interesting to note that C's realloc() function does not have such severe problems. The reason is that once the array is large enough, it will be allocated from the end of the heap. realloc can then expand the buffer by increasing the size of the memory block without having to copy the data. Unfortunately, this does not work with C++ because realloc cannot call constructors on the expanded memory. C++ requires us to copy the elements explicitly.

You might try to address these problems by growing the array by more than one element at a time. However, this does not significantly reduce the memory usage. Even though it reduces the total number of copies, it is still an O(N2) algorithm-all you've done is decrease the running constant. For example, if you add 1000 elements and grow the array by 10 elements whenever it's full, you are still performing 10+20+..+1000 = 50,500 copy operations, which is still O(N2). Although this is 10 times less copying than before, it is still 50 times more copying than is necessary.

HATs Usually Append in O(1) Time

To overcome the limitations of variable-length arrays, I created a data structure that has fast constant access time like an array, but mostly avoids copying elements when it grows. I call this new structure a "Hashed-Array Tree" (HAT) because it combines some of the features of hash tables, arrays, and trees.

Although used to implement one-dimensional arrays, HATs are really two-dimensional; see Figure 2. A HAT consists of a Top array and a number of Leaves which are pointed to by the Top array. The number of pointers in the Top array and the number of elements in each Leaf is the same, and is always a power of 2.

Because the Top and Leaf arrays are powers of 2, you can efficiently find an element in a HAT using bit operations; see Example 2. Usually, appending elements is very fast since the last leaf may have empty space. Less frequently, you'll need to add a new leaf, which is also very fast and requires no copying.

When the Top array is full, it becomes a bit more interesting. My implementation first computes the correct size (Top and Leaf arrays are the same size, both a power of 2), then copies the elements into a new HAT structure, freeing the old leaves and allocating new leaves as it goes.

This approach dramatically avoids most of the element copying performed by the previous array implementation. Recopying only occurs when the Top array is full, and this only happens when the number of elements jusrt exceeds the square of a power of 2. If N=4n, then the total amount of recopying is 1+4+16+64+256+...+N. Using the identity *(x(n+1)-1)=(x-1) (1+x+x2+x3+... + xn)* you have *1 +4+42+43+...+4n = (4(n+1) -1)/(4-1) = (4N-1)/3*, or about 4/3 N. This means that the average number of extra copy operations is O(N) for sequentially appending N elements, not O(N2).

The technique of reallocating and copying each leaf one at a time also significantly reduces the memory overhead. Instead of needing memory for a complete copy of the HAT, you only need enough memory for an extra leaf. Because the leaf size depends on the square root of N, the extra memory required during resizing decreases dramatically as a percentage of N. For example, if you added 1,000,000 elements to the HAT, the extra memory needed for the last resize would be 1024 elements or about 0.1 percent of the memory already used for the array's data.

In addition, memory fragmentation is reduced. Since the Top and Leaf sizes always increase to the next power of 2, the heap manager may be able to combine two freed leaves from the smaller HAT to allocate a leaf in the larger one.

Available electronically is code for a HAT template that includes some further optimizations to eliminate copying when the HAT grows and shrinks across resize boundaries. This is achieved at some expense of potential memory fragmentation, but results in smoother performance. The implementation allows the Top array to increase as a multiple of its former ideal size until an up or down threshold is reached.

Even if resizing is not an issue, this implementation has other advantages. For example, in a 16-bit environment, no object on the heap can exceed 64 KB. A HAT can manage arrays much larger than this without any single leaf exceeding 64 KB. You could also extend HATs to automatically swap leaves to and from disk storage to support truly huge arrays with a minimal memory footprint. Finally, there may be advantages to extending HATs to three or more levels rather than the two levels I've used.

Memory Overhead

I've already suggested that HATs use much less memory than the standard approach of reallocating and copying the entire array. To see how much less, I'll compute the actual memory overhead of a HAT resize.

The worst case happens when the elements in the HAT are the same size as pointers and the number of elements is one greater than a resize value. If the elements are the same size as pointers, you can add the unused portion of the Top array to any wasted memory in the Leaves, thus maximizing the amount of wasted memory as a percentage of the data in the array; see Table 1. You can see that as the HAT grows, the percentage of wasted memory in the worst case drops dramatically. Generally, the worst-case memory waste is *(top+leaf-1) ~= 2*sqrt(N) = O(sqrt(N))*. If you expect the last leaf to be half full, the expected memory waste drops to *(top + leaf/2) ~= 1.5*sqrt(N), which is still O(sqrt(N))*. This overhead compares well with other data structures that can add elements in expected *O(1)* time. For example, singly linked lists require O(N) memory overhead (one pointer for each element).

Performance

In a built-in C++ array, the address of the element is computed by adding the index of the element, times its size, to the start of the array. By comparison, finding the address of an element in a HAT requires two such array lookups, a pointer dereference, and the topIndex and leafIndex calculations. By precomputing (1power)-1, this becomes two array lookups, a pointer dereference, a bit shift, and an & operation. On modern processors, the last three can be done very quickly. This means that HAT elements can be dereferenced in about twice the time required for a C++ array.

Figure 3 shows the results of a quicksort implementation comparing standard C++ arrays and HATs. This algorithm uses array references very heavily, and supports the estimate that HATs are about twice as slow as regular arrays.

Figure 4 compares the two types of arrays under slightly different circumstances. In this case, N random integers are appended to the array and then sorted. Even though the HAT is slower for array references, the faster append makes the overall operation of this test significantly faster.

The benchmarks compare HATs to a resizable-array class implemented in the standard way (reallocate and copy) that grows by 64 elements whenever it runs out of space. The quicksort algorithm is a template that accepts either resizable-array class. All benchmarks were run on a 100Mhz HP 700 series PA-RISC workstation running HPUX 9.01 with 256 MB of memory.

Conclusion

HATs are a practical and efficient way to implement variable-length arrays. They offer highly desirable O(N) performance to add N elements to an empty array and only require O(sqrt(N)) memory overhead. They provide all the ease of use and

standard features of normal arrays, including random access to elements. They maintain their performance and memory-usage characteristics over any number of elements and require no special application performance tuning.

Although Hashed-Array Trees require more time to access their elements, this drawback is often far outweighed by their fast resizing, efficient memory utilization, and high programmer convenience. They offer strong advantages over other methods of implementing variable-length arrays.

References

Cline, M.P. and G.A. Lomow, C++ FAQs, Reading, MA: Addison-Wesley, 1995.

Cormen, T.H., C.E. Leiserson, and R.L. Rivest. Introduction to Algorithms, Cambridge, MA: MIT Press, 1990.

Example 1: A bad way to resize an array.

```
void vArray::append( T &el )
{
     // Extremely bad implementation of
     // variable length array.
     T *newTArray = new T [numElements+1];
     for(size_t j = 0; j < numElements; j++)
         newTArray[j]  =     tArray[j];
     delete tArray;
     tArray = newTArray;
     tArray[maxElements++] = el;
}
```

Example 2: Fast indexing with bit operations.

```
inline size_t Hat::topIndex(const size_t j ) const
{
    // Get the high index.
    return j  power;
}
inline size_t Hat::leafIndex(const size_t j ) const
{
    // Get the low index.
    return j & ((1power)-1);
}
inline T &Hat::operator[](const size_t j) const
{
    // Return an element in the HAT. Do no bounds checking.
    return top[topIndex(j)][leafIndex(j)];

}
```

Table 1: Worst-case memory overhead.

Top/Leaf Size	Worst N	Worst Waste (top+leaf-1)	Percent of Data
2	3	1+2=3	100
4	5	4+3=7	140
8	17	8+7=15	80
16	65	16+15=31	48
32	257	32+31=63	25
64	1025	64+63=127	8
128	4097	128+127=255	6
256	16385	256+255=511	3

Figure 1: Memory usage during array resizing.

Figure 2: HAT data structure.

Figure 4: Append and Sort with standard arrays and Hats.

Figure 3: QuickSort with standard arrays and HATs.

Previous Page | 1 | 2 | 3 | 4 | 5